MSARCH – Project Specifications

GPU-Accelerated Network Screening for Alzheimer's Disease
Drug Repurposing, with an Optional Quantum Drug-Combination Solver

September 2026

Abstract

Developing a new Alzheimer's disease (AD) drug takes more than a decade, and almost all candidates fail. *Drug repurposing*—finding already-approved drugs that might also help AD—is a faster route. A well-established way to screen for such drugs treats the cell's proteins as a graph: a drug is a promising candidate if the proteins it acts on sit unusually close, in shortest-path distance, to the proteins linked to AD. Deciding what "unusually close" means requires comparing each drug against thousands of random look-alikes, which makes the screen expensive. We propose a CUDA implementation of this screen. It computes all shortest-path distances on the human protein-interaction network with a bit-parallel breadth-first search, then scores every drug against thousands of random samples in parallel on the GPU. We compare its execution time against a sequential C++ version, check that both produce identical distances, and test whether the resulting ranking places drugs that have already entered AD clinical trials near the top. As an optional extension, we express the choice of a small drug combination as a Quadratic Unconstrained Binary Optimization (QUBO) problem and compare an exact GPU brute-force solver with the Quantum Approximate Optimization Algorithm (QAOA) simulated on the GPU with NVIDIA cuQuantum.

1 Introduction

1.1 Why Alzheimer's disease needs faster drug discovery

Alzheimer's disease (AD) is the most common cause of dementia. In the brain it shows two hallmarks: clumps of a protein fragment called amyloid-β that build up between nerve cells (plaques), and twisted fibers of a protein called tau inside nerve cells (tangles). Over years, nerve cells stop working and die, and memory and thinking decline.

Developing a brand-new drug from scratch takes more than ten years, and AD has one of the highest failure rates of any disease area. **Drug repurposing** offers a shortcut: take a drug that is already approved for something else (so its safety in humans is known) and test whether it could also help AD. The hard part is choosing which of the thousands of existing drugs deserve a closer look. That choice is what computation can help with.

1.2 Thinking of the cell as a graph

Proteins are the molecular machines that do most of the work in a cell, and they rarely work alone: they bind to and act on each other. If we draw every human protein as a node and every known physical interaction between two proteins as an edge, we get the **human protein–protein**

interaction network, or *interactome*: a graph with roughly 15,000–20,000 nodes and hundreds of thousands of edges.

Two ideas from *network medicine* [1] make this graph useful:

1. Proteins linked to the same disease tend to cluster in the same neighborhood of the graph, forming a *disease module* [2]. For AD, the starting points are genes identified by large genetic studies [3].
2. Most drugs work by binding to one or a few proteins, called the drug's *targets*. A drug whose targets sit close to the disease module is more likely to affect the disease [4].

In plain words

Imagine a city map where every building is a protein and every road is an interaction. AD "lives" in one neighborhood. A drug is a delivery truck that stops at a few specific buildings. If those stops are only a street or two from the AD neighborhood, the drug might influence it. If they are across town, it probably won't. The screen measures the distance from each drug's stops to the AD neighborhood.

This approach has already produced testable AD hypotheses: network screening combined with analysis of patient insurance records pointed to sildenafil as an AD candidate [5], and recent pipelines rank drugs for different stages of AD [6] or also screen out drugs with heart-rhythm risks [7].

1.3 Why this is a computer architecture problem

"Close" has to be judged fairly. Hub proteins with many connections are close to almost everything, so a raw distance would favor drugs that simply hit popular proteins. The standard fix [4] is to compare each drug against many random "fake drugs" whose targets have similar numbers of connections, and report how unusual the real distance is as a z-score. This creates a lot of work:

- shortest-path distances between every pair of proteins: one breadth-first search (BFS) from each of ∼18,000 nodes;
- for each of several thousand drugs, 1,000 or more random samples, each needing many distance look-ups;
- for drug combinations, the same distances between every pair of drugs.

All of these are *discrete* and *data-parallel*: thousands of independent BFS runs, and millions of independent random samples that each do the same look-up-and-minimum operation. That is exactly the shape of work GPUs are built for. Commonly used tools, however, are single-threaded Python scripts, which limits how many random samples, network versions, and disease modules researchers can afford to test.

1.4 An optional quantum extension

Once single drugs are ranked, a natural next question is which small *combination* of drugs covers the disease module well without the drugs overlapping each other [8]. Choosing the best subset from a shortlist is a yes/no decision per drug, which can be written as a Quadratic Unconstrained Binary Optimization (QUBO) problem [9, 10]. QUBOs are the standard input for quantum optimization algorithms such as QAOA [11], which we can simulate on the GPU with NVIDIA cuQuantum [12]. Because the shortlist is small, we can also find the exact best answer by brute force on the GPU and measure how close QAOA gets.

1.5 What this project delivers

1. A CUDA program that computes all shortest-path distances on the human interactome and scores every drug's closeness to the AD module.
2. A sequential C++ version of the same steps, and an execution-time comparison between the two.
3. A ranked list of repurposing candidates for AD, checked against drugs that have already entered AD clinical trials.
4. (Optional) A QUBO formulation of AD drug-combination selection, solved exactly on the GPU and approximately with simulated QAOA.

Section 2 reviews related work, Section 3 states the problem and objectives, and Sections 4–6 follow the course outline: project description, target platform, and milestones. Technical details are collected in the appendices.

2 Review of Related Literature

2.1 Network medicine and the disease module

Network medicine studies disease through the network of molecular interactions rather than one gene at a time [1]. Menche et al. [2] showed that proteins associated with the same disease form a connected neighborhood (a *disease module*) in the human interactome, even though the interactome is still incomplete. They also defined a *separation* measure: two sets of proteins that overlap in the network have negative separation, and sets that are far apart have positive separation.

2.2 Network proximity for drug repurposing

Guney et al. [4] turned this into a drug screen. For a drug with target set T and a disease module S, the *closest distance* is the average, over the drug's targets, of the shortest-path distance to the nearest disease protein. To correct for hub proteins, they compare it with the same measure for random target and disease sets that have matching degrees, giving a z-score. They showed that drugs with significantly negative z (closer than chance) are enriched for drugs that actually treat the disease. Cheng et al. [13] applied the approach to cardiovascular disease and confirmed predictions with patient-record data, and later showed that good drug combinations follow a *complementary exposure* pattern: both drugs are close to the disease module, but their own targets are separated from each other [8].

2.3 Network proximity studies in Alzheimer's disease

Fang et al. [5] built AD modules around amyloid and tau biology, screened drugs by proximity, and used insurance claims data to find that sildenafil users had lower AD incidence. A 2023 pipeline [6] built separate modules for different pathological stages of AD and ranked candidate drugs for each, and a 2025 study [7] combined proximity ranking with screening for drugs that raise the risk of a dangerous heart rhythm (Long QT syndrome). These studies establish both the method and its relevance to AD. The inputs they rely on are public: interaction databases such as BioGRID [14] and STRING [15], drug–target databases such as ChEMBL [16], DGIdb [17], and DrugBank [18], AD genes from genome-wide association studies [3], and the annual AD drug-development pipeline review [19], which lists drugs currently in AD clinical trials.

2.4 Graph traversal on GPUs

Breadth-first search is the building block for shortest paths on unweighted graphs. Harish and Narayanan [20] gave one of the first CUDA BFS implementations, Merrill et al. [21] showed how to make GPU BFS efficient with prefix-sum-based frontier expansion, and Gunrock [22] provides a general GPU graph library. When many BFS runs share one graph, *multi-source BFS* (MS-BFS) [23] runs up to 64 or more searches at once by storing, for each node, a bit per search; expanding the frontier then becomes bitwise OR and AND on machine words. This fits data-level parallelism well: each thread handles one node and one word of bits, with no branching per search.

2.5 QUBO, quantum optimization, and drug combinations

A QUBO asks for the binary vector x that minimizes a quadratic function $x^\top Qx$. Many selection problems can be written this way [9, 10]. QUBOs can be attacked with classical heuristics such as simulated annealing [24], with quantum annealers, or with the gate-based QAOA [11]. NVIDIA cuQuantum [12] simulates quantum circuits on GPUs, which lets QAOA be tested on small problems without quantum hardware. Most relevant here, a 2026 study [25] formulated network-based drug-combination prediction as a QUBO and solved it with quantum annealing.

2.6 Synthesis and gap

Work	What it does	AD	GPU	Quantum
Guney et al. [4]	Proximity z-score drug screen	–	–	–
Cheng et al. [8]	Complementary-exposure drug combinations	–	–	–
AD proximity studies [5, 6, 7]	AD drug repurposing	✓	–	–
GPU BFS / MS-BFS [21, 23, 22]	Fast graph traversal	–	✓	–
Quantum drug combinations [25]	QUBO + quantum annealing	–	–	✓
This work	GPU proximity screen + optional QAOA combinations	✓	✓	✓

Table 1: How this project relates to prior work. [TODO: Confirm from the papers whether any of the AD studies or [25] used GPUs or targeted AD.]

The method (network proximity), its relevance to AD, fast GPU graph traversal, and quantum formulations of drug combinations all exist, but separately. Proximity screens are typically run as sequential scripts, which caps the number of random samples and the number of network and disease-module variants that can be compared. We found no GPU implementation of the full proximity screen, and no study comparing exact and simulated gate-model QAOA solutions for AD drug combinations.

3 Problem Statement

3.1 Problem

Network proximity is a proven way to shortlist drugs for repurposing, and it has been applied to Alzheimer's disease. But the screen is expensive to run properly. It needs shortest-path distances between all pairs of ∼18,000 proteins, and each of several thousand drugs must be compared against 1,000 or more random samples to get a fair z-score. Typical implementations are sequential scripts,

so researchers settle for few random samples, one version of the interaction network, and one definition of the AD module—even though results can change with each of these choices.

A second, smaller problem sits on top: choosing a good *combination* of drugs from the shortlist is a combinatorial selection problem. It can be written as a QUBO, the input format for quantum optimization, but it is not yet known how well a gate-based quantum algorithm (QAOA) handles this specific AD problem compared with the exact answer.

3.2 Proposed solution

We propose a CUDA program that runs the full proximity screen on the GPU, built as five modules with clear inputs and outputs so each can be developed and tested separately (Table 2).

Module	Input	Output	Runs on
1. Data preparation	Interaction list, drug–target list, AD gene list	Graph in compressed form (CSR), target and disease sets as node IDs	CPU
2. All-pairs distances	Graph (CSR)	Distance matrix D ($N \times N$, 1 byte per entry)	GPU
3. Proximity scoring	D, drug target sets, AD module	Proximity and z-score per drug	GPU
4. Ranking and validation	z-scores, list of AD trial drugs	Ranked candidates, validation metrics	CPU
5. Combinations (optional)	Top-n drugs, D	Drug–drug separation matrix, QUBO, best combination	GPU (+ cuQuantum)

Table 2: Pipeline modules. Module 5 is optional and independent of the graded core (Modules 1–4).

3.3 Research questions

RQ1. How much faster is the GPU implementation than a sequential C++ implementation for the all-pairs distance step and the proximity scoring step, and how does the speedup change with network size and number of random samples?

RQ2. Does the GPU implementation give exactly the same distances, and the same z-scores when given the same random samples, as the sequential version?

RQ3. Does the proximity ranking place drugs that have entered AD clinical trials higher than simple baselines (random ranking, or ranking by number of targets)?

RQ4. (Optional) For AD drug-combination QUBOs small enough to solve exactly, how close does simulated QAOA get to the true best answer?

3.4 Objectives

General objective. To build and evaluate a GPU-accelerated network proximity screen for Alzheimer's disease drug repurposing.

Specific objectives.

1. Implement all-pairs shortest-path distances on the GPU using bit-parallel multi-source BFS.
2. Implement proximity and z-score computation for all drugs on the GPU, with random sampling done in parallel.
3. Implement the same steps sequentially in C++ and compare execution times.
4. Verify that GPU and CPU results match.
5. Rank AD repurposing candidates and measure how well the ranking recovers drugs already in AD trials.
6. (Optional) Formulate AD drug-combination selection as a QUBO and compare GPU brute force, simulated annealing, and QAOA.

3.5 Scope and limitations

- Public data only: interaction, drug–target, and AD gene databases. No patient data are used.
- Single GPU. The network has up to about 20,000 nodes, so the full distance matrix (at most 400 MB at one byte per entry) fits in GPU memory.
- Results are *hypotheses*, not treatment recommendations. Closeness in a network does not prove a drug works; clinical evidence is outside the project's scope.
- The interactome is incomplete and biased toward well-studied proteins; we reduce this by comparing more than one database.
- The quantum part is simulation only, limited to about 20–30 drugs (one qubit each). It does not claim to be faster than classical methods; its purpose is to measure solution quality.

3.6 Significance

A fast screen lets researchers run the checks that are currently skipped: more random samples for stable z-scores, several networks, and several definitions of the AD module (for example, early-stage vs. late-stage genes). For the course, it is a clear case study of turning a graph problem into bitwise, data-parallel GPU work. For the optional part, it provides a realistic, biology-derived benchmark for quantum optimization.

4 Methodology

This section follows the course's expanded outline. Formulas, kernel pseudocode, and the full evaluation protocol are in Appendices A–E.

4.1 Description of the Proposed Project

4.1.1 Inputs

All inputs are public and can be downloaded at the start of the project.

- **Human protein–protein interaction network.** Physical interactions between human proteins from BioGRID [14], and, for comparison, STRING [15] at several confidence cutoffs. We keep the largest connected piece of the graph: $N \approx$ 15,000–20,000 nodes and E in the hundreds of thousands of edges. Edges are unweighted and undirected.

- **Drug–target list.** For each drug, the set of proteins it binds, from ChEMBL [16] and DGIdb [17] (DrugBank [18] optionally, under its free academic license). We keep drugs with at least one target in the network: roughly 2,000–5,000 drugs.

- **AD disease module.** Genes linked to AD by the largest recent genetic study [3], plus well-known AD genes (e.g., *APP*, *PSEN1*, *PSEN2*, *APOE*, *MAPT*). About 50–150 proteins after mapping to the network.
- **Validation list.** Drugs that have entered AD clinical trials, from the annual AD drug-development pipeline review [19] and ClinicalTrials.gov.
- **Parameters.** Number of random samples per drug $R \in \{100, 1{,}000, 10{,}000\}$, and, for the optional part, shortlist size $n \leq 28$ and combination size k.

In plain words

The GPU only ever sees three things: a graph (who connects to whom), a few thousand small lists of node IDs (each drug's targets), and one more list of node IDs (the AD module). Everything biological is just labels on those node IDs.

4.1.2 Proposed Process

The screen answers one question for every drug: *are this drug's targets closer to the AD module than we would expect by chance?*

1. **Distance.** For each target of the drug, find the shortest path (fewest edges) to the nearest AD protein, and average these over the drug's targets. This is the drug's *proximity* d; smaller means closer.
2. **Chance level.** Repeat the same calculation R times using random "fake" target sets and random "fake" AD modules of the same sizes, where each random protein is picked to have a similar number of connections as the real one it replaces. This gives the average μ and spread σ of proximity by chance.
3. **Score.** $z = (d - \mu)/\sigma$. A strongly negative z (for example $z < -2$) means the drug is unusually close to AD.

(i) Processes parallelized with SIMT (GPU/CUDA).

1. **All-pairs shortest-path distances (Module 2).** One BFS from every node gives the full distance matrix D, where $D[a][b]$ is the number of steps from protein a to protein b. Since distances in this network are small (rarely above 15), each entry fits in one byte. We run many BFS searches at once using *bit-parallel multi-source BFS* [23]: each node keeps one bit per search, so a 64-bit word tracks 64 searches, and one step of all 64 searches becomes a bitwise OR over the node's neighbors.
2. **Proximity and random sampling (Module 3).** Every (drug, random sample) pair is independent. Each GPU thread generates one random sample, looks up the needed distances in D, takes minimums and an average, and a block-level reduction combines the samples into μ and σ for that drug.
3. **Drug–drug separation (Module 5).** For the shortlist, the distance-based separation between every pair of drugs, computed with the same look-up-and-minimum pattern.
4. **Exact QUBO solver (Module 5, optional).** Tries all 2^n yes/no choices for n shortlisted drugs, one GPU thread per group of choices; practical up to $n \approx 30$.

5. **QAOA simulation (Module 5, optional).** Simulated with cuQuantum's cuStateVec, which itself runs data-parallel over the 2^n numbers that describe the quantum state.

(ii) Related works and how this implementation differs. Network proximity for drug repurposing was introduced by Guney et al. [4] and has been applied to AD several times [5, 6, 7]. These studies focus on the biology and generally run the screen as sequential scripts. GPU BFS is well studied [20, 21, 22], and bit-parallel MS-BFS was designed for running many searches over one graph [23], but these are general-purpose graph tools, not proximity screens. Quantum drug-combination selection was recently formulated as a QUBO and solved with quantum annealing [25].

Our implementation differs in that it:

- runs the *whole* screen on the GPU, including the random sampling, which is usually the slowest part;
- stores distances as one byte each and produces them with bitwise MS-BFS, so the full matrix for 20,000 proteins (400 MB) stays on the GPU and every later step is a fast memory look-up;
- uses the same counter-based random number generator on CPU and GPU, so both versions draw identical random samples and their results can be compared exactly;
- targets AD specifically and checks results against AD trial drugs;
- (optional) compares gate-based QAOA against the exact optimum on the same AD instances, instead of reporting a quantum solution alone.

Section 2 gives the full review.

(iii) Parallelization plan.

Step	How work is split across GPU threads	Limited by
Multi-source BFS	Sources processed in batches of $64 \times W$ (e.g., 1,024). One thread per (node, 64-bit word). Each level: OR the neighbors' frontier words, AND with NOT-visited, write the level number into D for newly reached sources. Graph stored in CSR form. Repeat until no bits change.	Memory
Writing D	Threads for neighboring nodes write neighboring bytes of the same row of D, so writes are coalesced.	Memory
Proximity + z-score	One thread block per drug; each thread handles several random samples; degree-matched sampling from per-degree-bin node lists; distances read from D (reads are scattered, so L2 cache matters); block reduction for μ, σ.	Memory
Drug–drug separation	One thread per drug pair on the shortlist; same look-up pattern.	Memory
QUBO brute force	One thread per block of 2^m bit patterns; patterns visited in Gray-code order so each step flips one bit and updates the cost in $O(n)$; warp and block minimum reduction.	Compute

Data moves to the GPU once (the graph and ID lists, a few MB) and results come back once (z-scores; D only if needed). Pseudocode and memory layouts are in Appendix B.

(iv) Processes not parallelized.

- Downloading and reading the databases; converting between gene and protein identifiers; removing duplicates and self-loops.
- Building the CSR graph and keeping its largest connected component.
- Grouping proteins into degree bins for the random sampling.
- Ranking drugs, computing validation metrics, and plotting.

- Building the QUBO matrix from the separation results, and the classical optimizer loop that tunes QAOA's parameters.

4.1.3 Outputs

1. **Distance matrix** D for the interactome (binary file, one byte per entry), plus summary statistics (e.g., distance histogram).
2. **Ranked drug list** with proximity d, z-score, and p-value for every drug, and the top AD repurposing candidates.
3. **Execution-time comparison**: sequential C++ vs. CUDA time for (a) all-pairs distances and (b) proximity scoring, across network sizes N and random-sample counts R, with speedup charts.
4. **Correctness report**: GPU and CPU distance matrices are identical, and z-scores match when using the same random samples.
5. **Validation**: how highly AD trial drugs are ranked, compared with simple baselines (Appendix D).
6. **(Optional) Combination results**: best k-drug combinations from the shortlist, and QAOA's solution quality compared with the exact answer.

5 Proposed Target Implementation Platform

Component	Specification
Parallel paradigm	SIMT on NVIDIA GPUs via CUDA C++
GPU	[TODO: model]. Minimum: compute capability ≥ 7.0 and ≥ 4 GB memory (D needs at most 400 MB). For the optional QAOA part, ≥ 16 GB allows about 30 qubits.
CPU (sequential baseline)	[TODO: model, cores, RAM]; same machine as the GPU
OS	Linux (Ubuntu 22.04/24.04), or Windows with WSL2
GPU libraries	CUDA Toolkit 12.x, CUB (block reductions), a counter-based RNG (Philox, as in cuRAND/Random123) shared with the CPU code; cuQuantum (cuStateVec) through CUDA-Q for the optional part
Languages and build	C++17 and CMake for the core; Python 3.11+ (pandas, NumPy, SciPy, matplotlib) for data preparation, validation, and plots; NetworkX as an independent reference on small graphs
Profiling	Nsight Systems and Nsight Compute
Repository	GitHub: source, build and benchmark scripts, the discussion of the parallel algorithms, and the sequential-vs-parallel timing results required for Milestone 3

6 Mid-Project Milestones

Milestone 2 (Week 10) deliverable. A live demo on the BioGRID interactome that shows:

1. the GPU multi-source BFS producing the full distance matrix, and a check that it is identical to the sequential C++ result;
2. the GPU proximity and z-score step for all drugs with $R = 1{,}000$ random samples;
3. a first sequential-vs-parallel timing table for both steps;

4. a first ranked list of AD candidates.

The full schedule is in Table 3.

Week	Work
6	Proposal submission (Milestone 1). Download and freeze data versions. Set up repository and CMake build.
7	Data preparation (identifier mapping, CSR graph, degree bins). Sequential C++ BFS and proximity; check against NetworkX on a small graph.
8	GPU multi-source BFS; exact comparison with the CPU distance matrix.
9	GPU proximity and z-score kernel with shared RNG; timing harness (CUDA events, median of 10 runs).
10	**Milestone 2: mid-project demo** (above).
11	Scaling experiments (STRING cut-offs, R up to 10,000); profiling; validation against AD trial drugs and baselines.
12	(Optional) Drug–drug separation, QUBO, GPU brute force, simulated annealing, and QAOA via CUDA-Q.
13	**Milestone 3**: final GitHub submission with source, parallel-algorithm discussion, and sequential-vs-parallel timing; demo; optional conference paper.

Table 3: Project schedule by academic week.

Fallback. If the optional quantum part is not ready by Week 12, it is reported as future work. The graded core (Modules 1–4) does not depend on it.

References

[1] A.-L. Barabási, N. Gulbahce, and J. Loscalzo, "Network medicine: a network-based approach to human disease," *Nature Reviews Genetics*, vol. 12, no. 1, pp. 56–68, 2011.

[2] J. Menche, A. Sharma, M. Kitsak, S. D. Ghiassian, M. Vidal, J. Loscalzo, and A.-L. Barabási, "Uncovering disease-disease relationships through the incomplete interactome," *Science*, vol. 347, no. 6224, p. 1257601, 2015.

[3] C. Bellenguez, F. Küçükali, I. E. Jansen, L. Kleineidam, S. Moreno-Grau, N. Amin, *et al.*, "New insights into the genetic etiology of Alzheimer's disease and related dementias," *Nature Genetics*, vol. 54, pp. 412–436, 2022.

[4] E. Guney, J. Menche, M. Vidal, and A.-L. Barabási, "Network-based in silico drug efficacy screening," *Nature Communications*, vol. 7, p. 10331, 2016.

[5] J. Fang, P. Zhang, Y. Zhou, C.-W. Chiang, J. Tan, Y. Hou, *et al.*, "Endophenotype-based in silico network medicine discovery combined with insurance record data mining identifies sildenafil as a candidate drug for Alzheimer's disease," *Nature Aging*, vol. 1, pp. 1175–1188, 2021.

[6] TODO: authors, "Network proximity-based computational pipeline identifies drug candidates for different pathological stages of Alzheimer's disease," *Computational and Structural Biotechnology Journal*, 2023.

[7] TODO: authors, "Prioritizing repurposable drugs for Alzheimer's disease using network-based analysis with concurrent assessment of Long QT syndrome risk." `https://pmc.ncbi.nlm.nih.gov/articles/PMC12337645/`, 2025. PMID: 40791766.

[8] F. Cheng, I. A. Kovács, and A.-L. Barabási, "Network-based prediction of drug combinations," *Nature Communications*, vol. 10, p. 1197, 2019.

[9] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics I: a tutorial on formulating and using QUBO models," *4OR*, vol. 17, pp. 335–371, 2019.

[10] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[11] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm." arXiv preprint arXiv:1411.4028, 2014.

[12] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, *et al.*, "cuQuantum SDK: A high-performance library for accelerating quantum science," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023. arXiv:2308.01999.

[13] F. Cheng, R. J. Desai, D. E. Handy, R. Wang, S. Schneeweiss, A.-L. Barabási, and J. Loscalzo, "Network-based approach to prediction and population-based validation of in silico drug repurposing," *Nature Communications*, vol. 9, p. 2691, 2018.

[14] R. Oughtred, J. Rust, C. Chang, B.-J. Breitkreutz, C. Stark, A. Willems, *et al.*, "The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions," *Protein Science*, vol. 30, no. 1, pp. 187–200, 2021.

[15] D. Szklarczyk, R. Kirsch, M. Koutrouli, K. Nastou, F. Mehryary, R. Hachilif, *et al.*, "The STRING database in 2023: protein-protein association networks and functional enrichment analyses for any sequenced genome of interest," *Nucleic Acids Research*, vol. 51, no. D1, pp. D638–D646, 2023.

[16] B. Zdrazil, E. Felix, F. Hunter, E. J. Manners, J. Blackshaw, S. Corbett, *et al.*, "The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1180–D1192, 2024.

[17] S. L. Freshour, S. Kiwala, K. C. Cotto, A. C. Coffman, J. F. McMichael, J. J. Song, *et al.*, "Integration of the Drug–Gene Interaction Database (DGIdb 4.0) with open crowdsource efforts," *Nucleic Acids Research*, vol. 49, no. D1, pp. D1144–D1151, 2021.

[18] C. Knox, M. Wilson, C. M. Klinger, M. Franklin, E. Oler, A. Wilson, *et al.*, "DrugBank 6.0: the DrugBank knowledgebase for 2024," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1265–D1275, 2024.

[19] J. Cummings, Y. Zhou, G. Lee, K. Zhong, J. Fonseca, and F. Cheng, "Alzheimer's disease drug development pipeline: 2024," *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, vol. 10, no. 2, p. e12465, 2024.

[20] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the GPU using CUDA," in *High Performance Computing – HiPC 2007*, vol. 4873 of *Lecture Notes in Computer Science*, pp. 197–208, 2007.

[21] D. Merrill, M. Garland, and A. Grimshaw, "Scalable GPU graph traversal," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pp. 117–128, 2012.

[22] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016.

[23] M. Then, M. Kaufmann, F. Chirigati, T.-A. Hoang-Vu, K. Pham, A. Kemper, T. Neumann, and H. T. Vo, "The more the merrier: Efficient multi-source graph traversal," *Proceedings of the VLDB Endowment*, vol. 8, no. 4, pp. 449–460, 2014.

[24] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[25] TODO: authors, "Network-based prediction of drug combinations with quantum annealing," *Quantum Machine Intelligence*, 2026.

A Formal Definitions

Let $G = (V, E)$ be the interactome with $N = |V|$ nodes, and $d(u, v)$ the shortest-path length (number of edges) between u and v.

Closest distance. For a drug with target set T and disease module S [4]:

$$d_c(S, T) = \frac{1}{|T|} \sum_{t \in T} \min_{s \in S} d(s, t). \tag{1}$$

Degree-matched randomization. Nodes are grouped into bins of similar degree, each bin holding at least $b_{\min}$ nodes (e.g., 100). A random sample replaces every node of T and of S by a uniformly drawn node from the same bin. Over R samples, with mean μ and standard deviation σ of d_c,

$$z = \frac{d_c(S, T) - \mu}{\sigma}, \tag{2}$$

with an empirical one-sided p-value $p = (1 + \#\{r : d_c^{(r)} \leq d_c\})/(1 + R)$.

Separation. For two node sets A and B [2],

$$s_{AB} = \langle d_{AB} \rangle - \frac{\langle d_{AA} \rangle + \langle d_{BB} \rangle}{2}, \tag{3}$$

where $\langle d_{AA} \rangle$ is the mean over $a \in A$ of the distance to the nearest *other* member of A, and $\langle d_{AB} \rangle$ is the mean, over all nodes of A and B, of the distance to the nearest node of the other set. $s_{AB} < 0$ means the two target sets overlap in the network.

Complementary exposure. A drug pair is favored when both drugs are close to the disease ($z < 0$) and their target sets are separated ($s_{AB} \geq 0$) [8].

Combination QUBO (optional). For a shortlist of n drugs with $x_i = 1$ if drug i is chosen, let $w_i = -z_i > 0$ reward disease proximity and $P_{ij} = \max(0, -s_{ij})$ penalize overlapping target sets. Choosing k drugs:

$$\min_{x \in \{0,1\}^n} \; -\sum_i w_i x_i + \lambda \sum_{i<j} P_{ij} x_i x_j + A\Big(\sum_i x_i - k\Big)^2. \tag{4}$$

Expanding with $x_i^2 = x_i$ gives $x^\top Q x$ (upper-triangular form) plus the constant Ak^2, with

$$Q_{ii} = -w_i + A(1 - 2k), \qquad Q_{ij} = \lambda P_{ij} + 2A \quad (i < j). \tag{5}$$

A must be large enough that violating the cardinality constraint never pays off (e.g., $A > \max_i w_i + \lambda \sum_j P_{ij}$); λ is swept to show the trade-off between proximity and overlap.

B Kernel Design Details

B.1 Data layout

- Graph: CSR arrays `rowptr` ($N + 1$ ints) and `col` ($2E$ ints).
- Distance matrix: D as `uint8_t`, row-major, N^2 bytes (324 MB at $N = 18{,}000$); 255 marks "unreachable".
- MS-BFS state per batch: `seen`, `frontier`, `next`, each $N \times W$ 64-bit words ($W = 16$ gives 1,024 sources per batch; 3 arrays $\times$ 18,000 $\times$ 16 $\times$ 8 B $\approx$ 7 MB).
- Target sets and the disease module: flattened index arrays with offsets; degree bins: node lists per bin, plus a node-to-bin map.

B.2 Bit-parallel multi-source BFS

Algorithm 1 One batch of MS-BFS (sources $s_0, \ldots, s_{64W-1}$)

```
seen[s_j] ← frontier[s_j] ← bit j set; D[s_j][s_j] ← 0
for ℓ = 1, 2, ... do
    for all nodes v and words w in parallel do                ▷ one thread per (v, w)
        m ← 0
        for all u ∈ neighbors(v) do
            m ← m | frontier[u][w]
        end for
        next[v][w] ← m & ¬seen[v][w]
        for all set bits j of next[v][w] do
            D[s_{64w+j}][v] ← ℓ
        end for
    end for
    seen ← seen | next;  swap(frontier, next)
    if no bit was set this level then break
    end if
end for
```

This is a "pull" formulation: each thread reads its neighbors' words and writes only its own, so no atomics are needed. Threads for consecutive v and the same w write consecutive bytes of

row s_{64w+j} of D, which coalesces. Iterating set bits uses `__ffsll` and clearing the lowest bit. High-degree nodes cause load imbalance; if profiling shows this, nodes are sorted by degree and heavy nodes are assigned a warp each. The number of levels equals the largest distance reached in the batch (typically ≤ 15).

B.3 Proximity kernel

One block per drug; thread t processes samples $r = t, t + B, t + 2B, \ldots$ For each sample it seeds a Philox counter-based generator with (drug ID, sample ID), so the CPU reference reproduces the exact same random nodes. It draws the random target and disease sets from the degree bins, evaluates d_c with reads from D, and accumulates $\sum d_c$ and $\sum d_c^2$ in registers. A CUB block reduction gives μ and σ. Reads from D are scattered; the real disease module's $|S|$ rows (e.g., 100 $\times$ 18,000 bytes $\approx$ 1.8 MB) fit in L2 cache, which helps the non-random term.

B.4 Exact QUBO solver

Thread t owns the 2^m patterns whose top $n - m$ bits equal t. It evaluates the first pattern in full ($O(n^2)$), then walks the rest in Gray-code order: flipping bit i changes the cost by $\Delta = (1 - 2x_i)\big(Q_{ii} + \sum_{j \neq i} c_{ij} x_j\big)$, where c_{ij} is the coefficient of $x_i x_j$ (i.e., Q_{ij} or Q_{ji}, whichever is in the upper triangle), which is $O(n)$, or $O(1)$ if a running vector of local fields is kept. Warp and block minimum reductions find the best pattern. At $n = 28$ this is 2.7×10^8 patterns.

B.5 Work estimates

Step	Sequential cost	Example size
All-pairs BFS	$O(N(N + E))$	$\approx 10^{10}$ edge visits (N=18k, E=500k)
Proximity, all drugs	$O(n_{\text{drug}} R \lvert T\rvert \lvert S\rvert)$	$\approx 1.5 \times 10^9$ look-ups (3k drugs, R=1k)
Separation, shortlist	$O(n^2 \lvert T\rvert^2)$	small
QUBO brute force	$O(2^n n)$	$\approx 7.5 \times 10^9$ at n=28

C Optional Quantum Part: QAOA with cuQuantum

Each shortlisted drug is one qubit. The QUBO is mapped to an Ising cost Hamiltonian H_C by $x_i = (1 - Z_i)/2$. QAOA of depth p prepares

$$|\psi(\gamma, \beta)\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell H_M} e^{-i\gamma_\ell H_C} |+\rangle^{\otimes n}, \qquad H_M = \sum_i X_i, \tag{6}$$

and a classical optimizer (COBYLA) tunes the $2p$ angles to minimize $\langle\psi|H_C|\psi\rangle$ [11]. The circuit is built in CUDA-Q and simulated with cuStateVec [12]; the expectation is computed exactly from the state vector, and the final state is sampled to read out combinations. A state of n qubits needs 2^n complex numbers: 2 GiB at $n = 28$ (complex64), 8 GiB at $n = 30$.

Reported for $p = 1, \ldots, 5$ and $n \in \{12, 16, 20, 24, 28\}$:

- approximation ratio $r = (E_{\text{worst}} - E_{\text{QAOA}})/(E_{\text{worst}} - E_{\text{opt}})$ over valid solutions, with E_{opt} from the GPU brute force;
- probability of sampling the optimum;
- fraction of samples violating the "exactly k drugs" constraint;
- comparison with simulated annealing [24] given the same number of cost evaluations.

No quantum speedup is claimed: at these sizes brute force is exact and fast. The goal is to measure QAOA solution quality on realistic, biology-derived instances.

D Evaluation Protocol

D.1 Timing

- Sequential baseline: single-threaded C++ implementing the identical algorithm (standard queue-based BFS from each node; proximity loop with the same RNG). An OpenMP version is reported as an extra baseline.
- Scaling: network size N varied through STRING confidence cut-offs (e.g., 900, 700, 400) and BioGRID; random samples $R \in \{100, 1{,}000, 10{,}000\}$.
- Protocol: two warm-up runs, then the median of 10. GPU time by CUDA events, reported both kernel-only and end-to-end including transfers. Speedup = $T_{\text{CPU}}/T_{\text{GPU}}$ per step and overall. Nsight Compute reports achieved memory bandwidth, cache hit rates, occupancy, and warp divergence.

D.2 Correctness

- $D_{\text{GPU}} = D_{\text{CPU}}$ exactly (byte-wise comparison); NetworkX as an independent check on small graphs.
- With shared Philox seeds, z-scores from GPU and CPU agree to floating-point rounding.
- Brute-force QUBO optimum checked against exhaustive CPU search for small n.

D.3 Biological validation

- **Positives:** drugs that have entered AD clinical trials [19]. **Metrics:** area under the ROC and precision–recall curves, and enrichment of positives in the top 50/100 (hypergeometric test).
- **Baselines:** random ranking; ranking by number of targets; ranking by target degree; ranking by direct overlap between targets and AD genes.
- **Circularity check:** many AD trial drugs target amyloid or tau directly, which are in the AD module. Metrics are also reported after removing drugs whose targets include AD-module genes, so the result is not just "drugs aimed at AD genes are close to AD genes."
- **Robustness (enabled by speed):** Spearman correlation of z-scores between BioGRID and STRING, across R, and across alternative AD modules (e.g., GWAS-only vs. GWAS plus known genes).

E Limitations and Risks

- **Incomplete network:** well-studied proteins have more known interactions; degree-matched sampling reduces but does not remove this bias.
- **Label quality:** being in a clinical trial does not mean a drug works; positives are "considered promising," not "effective."
- **Memory-bound kernels:** proximity look-ups are scattered, so speedup depends on cache behavior; this is measured, not assumed.
- **Amdahl's law:** data preparation in Python may dominate end-to-end time; it is reported separately.

- **Quantum part:** simulation only, $n \lesssim 30$; QAOA may return invalid or suboptimal combinations at low depth, which is reported as a result.
- **Not medical advice:** candidates are hypotheses for further study.